

بنام ایزد منان

پایان ترم ساختار کامپیوتر ترم 14042
استاد: دکتر سعید باقری شورکی گروه عصر

آرمین برزگری 402101352

سوال اول:

(الف) دلیل استفاده از معماری هاروارد و تأثیر آن در مسیر داده تک سیکل:

در پردازنده‌های با مسیر داده تک سیکل، تمام مراحل اجرای یک دستور (واکشی، رمزگشایی، اجرا، دسترسی به حافظه و نوشتن در ثبات) باید تنها در یک سیکل کلاک انجام شود. در دستورهای مانند LW (بارگذاری) یا SW (ذخیره)، پردازنده باید در یک سیکل، هم دستور را از حافظه بخواند و هم داده‌ای را از حافظه بخواند یا در آن بنویسد.

اگر از معماری فون‌نویمان (یک حافظه مشترک) استفاده می‌شد، برای اجرای چنین دستورهای به دو بار دسترسی به یک حافظه در یک سیکل نیاز داشتیم که باعث ایجاد مخاطره ساختاری (Structural Hazard) می‌شد. با انتخاب معماری هاروارد (جدا بودن کامل حافظه دستور و داده)، پردازنده می‌تواند در یک سیکل کلاک واحد، به طور همزمان دستور را از حافظه دستور واکشی کند و با حافظه داده تبادل اطلاعات انجام دهد. این انتخاب، طراحی مسیر داده را بسیار ساده می‌کند، زیرا نیاز به حافظه‌های پیچیده با چند پورت و یا متوقف کردن خط لوله را از بین می‌برد.

(ب) مشکل تغییر فلگ‌ها توسط دستور پرش در برنامه‌های شرطی:

در پردازنده PARS16، دستورهای پرش شرطی بر اساس فلگ‌هایی انجام می‌شوند که توسط دستورات قبلی (مثلاً یک دستور مقایسه مثل SUB با $S=1$) تنظیم شده‌اند. همچنین برای شبیه‌سازی شرط‌های معکوس (مثل شرط نامساوی NE که به طور مستقیم وجود ندارد)، پردازنده به الگوی دو دستوری نیاز دارد: یک پرش شرطی (B.EQ) که در صورت برقراری شرط از روی دستور بعدی می‌پرد، و بلافاصله پس از آن یک پرش بدون قید (B.AL) به مقصد واقعی.

اگر دستورهای پرش اجازه فعال سازی و تغییر فلگ ها را داشتند، اجرای پرش اول ممکن بود وضعیت فلگ ها را تغییر دهد. در نتیجه، اگر منطق برنامه نیازمند ارزیابی همان شرایط برای دستورات بعدی بود، فلگ ها مقدار اصلی و معتبر حاصل از عملیات محاسباتی را از دست می دادند و برنامه دچار خطای منطقی می شد. به بیان ساده، دستورهای پرش باید صرفاً مصرف کننده وضعیت فلگ ها باشند تا وضعیت شروط تخریب نشود.

ج) مدار محافظ رجیستر R0 و اثر حذف آن

همان طور که در جزوه اشاره شده، خواندن از ثبات R0 همیشه صفر می دهد و نوشتن روی آن همیشه بی اثر است که این کار توسط یک مدار ترکیبی کوچک انجام می شود.

- محل قرارگیری و عملکرد: این مدار دقیقاً در مسیر سیگنال فعال ساز نوشتن رجیستر فایل (پین Write Enable یا Rewrite) قرار دارد. این مدار شماره ثبات مقصد نوشتن (Rd) و سیگنال کنترلی RegWrite (تولید شده توسط واحد کنترل) را ارزیابی می کند.

- نحوه کار: مدار به گونه ای عمل می کند که وقتی آدرس مقصد برابر صفر (R0) باشد، سیگنال نهایی فعال سازی نوشتن در رجیستر فایل را خاموش (صفر) نگه می دارد، حتی اگر واحد کنترل دستور نوشتن داده باشد.

- اثر حذف مدار برای دستور ADD R0, R1, R2: اگر این مدار محافظ حذف شود، این دستور به صورت عادی اجرا می شود؛ یعنی مقادیر R1 و R2 با هم جمع شده و واحد کنترل سیگنال RegWrite را فعال می کند. در نتیجه، حاصل جمع واقعاً در ثبات R0 نوشته می شود. این اتفاق باعث می شود مقدار R0 از صفر تغییر کند و از آنجا که در این پردازنده (مثلاً در دستور CMP R1, R2 که مقصدش R0 است) فرض بر این است که نوشتن روی R0 بی اثر است و مقدار آن همیشه صفر می ماند، کل برنامه هایی

که به صفر بودن ذاتی R0 وابسته‌اند با خطای محاسباتی مواجه خواهند شد .

سوال دوم:

الف) تحلیل دستور SUB R2, R4, R1, S

سیگنال RegWrite: مقدار آن 1 است

چون این دستور از نوع محاسباتی است و با توجه به برقرار بودن شرط اجرا (بدون قید)، باید حاصل در رجیستر مقصد نوشته شود.

سیگنال RegDst: رجیستر R2

چون در دستورات فرمت R، آدرس مقصد نوشتن مستقیماً از فیلد Rd دستور استخراج می‌شود.

سیگنال‌های ورودی‌های A و B به ALU: مقدار A=10 و B=3

چون ورودی A همان Rs1 (یعنی R4) و ورودی B با توجه به صفر بودن ALUSrc، همان Rs2 (یعنی R1) است که از رجیسترفایل خوانده شده‌اند.

سیگنال خروجی ALU: مقدار 7 است چون حاصل تفریق 3 از 10 برابر 7 می‌شود.

سیگنال FlagWrite و مقادیر V,N,Z:

FlagWrite = 1

Z = 0

N = 0

$$V = 0$$

چون دستور دارای مشخصه S (فعال سازی فلگ) است ($s=1$) و شرط اجرا نیز برقرار است، FlagWrite یک می شود. خروجی 7 صفر نیست پس $Z=0$ است. خروجی مثبت است پس $N=0$ است. در تفریق، علامت R4 و R1 یکسان است (هر دو مثبت) بنابراین سرریز رخ نمی دهد و $V=0$ است. اگر $S=0$ باشد Flagwrite صفر می شود چون حاصل and دیگر 1 نخواهد بود.

ب) تحلیل دستور LW R5, 4(R3)

سیگنال MemWrite و MemRead: $MemRead = 1$ و $MemWrite = 0$

این دستور برای بارگذاری (خواندن) کلمه از حافظه داده (Data MEM) استفاده می شود و در حافظه چیزی نمی نویسد.

سیگنال ALUSrc و خروجی ALU: مقدار $ALUSrc = 1$ و خروجی $ALU = 24$

سیگنال $ALUSrc$ یک می شود تا مقدار فوری علامت گسترده شده (عدد 4) را انتخاب کند. خروجی ALU حاصل جمع آدرس پایه 20 و مقدار آفست 4 است و حاصل جمع $imm4$ و $Rs1$ که نهایتاً می شود 24.

سیگنال WBSeI: مقدارش 1 است

چون مالتی پلکسر باید داده ی خوانده شده از حافظه داده (Data MEM) را به جای خروجی ALU برای نوشتن در رجیستر فایل انتخاب کند.

سیگنال مقدار نهایی R5: مقدارش 99 است

چون داده ی موجود در آدرس محاسبه شده حافظه $MEM[24]$ در پایان سیکل درون رجیستر مقصد (R5) نوشته می شود

سوال سوم:

الف) فیلدهای مورد نیاز و فرمت مناسب برای دستور LWI
این دستور به دو فیلد ثبات نیاز دارد: یکی فیلد مقصد Rd برای ذخیره داده خوانده شده از حافظه، و دیگری فیلد مبدأ Rs1 که حاوی آدرس پایه است و باید در نهایت یک واحد افزایش یابد.

از میان فرمت‌های موجود، می‌توان از فرمت R یا فرمت I استفاده کرد.
دلیل این انتخاب آن است که در این دو فرمت، جایگاه مشخصی برای هر دو فیلد Rd و Rs1 تعبیه شده است. از آنجا که در این دستور به ثبات منبع دوم (در فرمت R) یا مقدار فوری (در فرمت I) نیازی نیست، می‌توان از این فیلدهای اضافه چشم‌پوشی کرد (مثلاً مقدار آن‌ها را توسط کامپایلر صفر در نظر گرفت).
فرمت‌های U و B/J به هیچ وجه قابل استفاده نیستند زیرا اصلاً فیلد Rs1 را در ساختار خود ندارند.

ب) پیشنهاد راهکار برای جا دادن LWI در فضای OPCODE:
با توجه به اینکه هر ۱۶ مقدار OPCODE اشغال شده‌اند، می‌توان از رویکرد دستور SYS (با آپکد 1111) استفاده کرد.
همان‌طور که دستور SYS با استفاده از یک فیلد فرعی به نام func دستورات NOP و HALT را از هم تفکیک می‌کند، می‌توانیم دستور جدید را تحت همان آپکد و با یک func جدید تعریف کنیم.
برای پیاده‌سازی، می‌توانیم آپکد 1111 را به دستور LWI اختصاص دهیم و در کنار استفاده از جایگاه‌های استاندارد Rd و Rs1، فیلد خالی انتهای دستور (معادل imm4 در فرمت I) را به عنوان فیلد func در نظر بگیریم.

با اختصاص یک کد جدید در این بخش (مثلاً $func = 0010$)، واحد کنترل می‌تواند بدون نیاز به افزایش عرض ۱۶ بیتی دستور، LWI را شناسایی و به درستی اجرا کند.

ج) کمبود و محدودیت مسیر داده فعلی:

در دستور LWI، ما به طور همزمان به دو عملیات نوشتن مجزا در ثبات‌ها نیاز داریم: ذخیره داده خوانده شده در Rd و ذخیره آدرس به‌روزشده ($Rs1 + 1$) در $Rs1$.

با بررسی مسیر داده در معماری فعلی مشخص می‌شود که رجیسترفایل تنها دارای یک پورت ورودی داده ($Data\ in$) و یک سیگنال فعال‌ساز نوشتن ($WriteEnable$) است.

بنابراین، این مسیر داده از نظر فیزیکی توانایی نوشتن در دو ثبات مختلف را در یک سیکل کلاک منفرد ندارد و این بزرگترین محدودیت برای اجرای تک‌سیکله دستور LWI است.

راه‌حل پیشنهادی برای رفع مشکل: افزودن سخت‌افزار برای حفظ اجرای تک‌سیکله

- باید یک پورت نوشتن دوم (شامل ورودی داده جدید، آدرس مقصد دوم و سیگنال $WriteEnable$ مجزا) به ساختار رجیسترفایل اضافه شود.
- علاوه بر آن، به یک جمع‌کننده ($Adder$) سخت‌افزاری اختصاصی نیاز است تا در موازات کار ALU ، مقدار $Rs1 + 1$ را محاسبه کرده و مستقیماً به پورت دوم نوشتن در رجیسترفایل متصل کند.

سوال چهارم:

الف) بررسی افزودن شرط NE و تغییر عرض فیلدها

چند بیت برای کد شرط لازم می شد؟ در حال حاضر ۴ حالت برای کد شرط (CC) در نظر گرفته شده است. (AL, EQ, LT, VS) با اضافه شدن شرط مستقل NE (نامساوی)، مجموع حالت‌های مورد نیاز به ۵ مورد می‌رسد.

برای نمایش ۵ حالت مجزا، به ۳ بیت نیاز داریم، زیرا با ۲ بیت تنها می‌توان ۴ حالت را نشان داد و برای پوشش ۵ حالت باید به توان بعدی ۲ برویم (5 بین دو به دو و دو به دو به توان سه است).

تأثیر بر عرض فیلد مقدار فوری (imm) در فرمت I :

طبق مشخصات فعلی، فرمت I از ۱۶ بیت به این شکل تشکیل شده است :

$$CC(2) + OPCODE(4) + Rd(3) + Rs1(3) + imm4(4) = 16.$$

اگر عرض فیلد CC از ۲ بیت به ۳ بیت افزایش یابد و عرض کل دستور (۱۶ بیت) و فیلد OPCODE (۴ بیت) ثابت بماند، این یک بیت اضافه باید از فیلد مقدار فوری (imm) کم شود.

محاسبه دقیق عرض جدید $3 = 16 - 13 = 16 - (3 + 4 + 3 + 3)$: بنابراین عرض جدید فیلد مقدار فوری به ۳ بیت کاهش می‌یابد.

ب) بررسی افزودن بیت S به تمامی فرمت‌ها:

*تأثیر اضافه شدن بیت S بر روی فرمت I و محاسبه عرض جدید:

اگر بخواهیم بیت S (1 بیت) را به تمامی فرمت‌ها از جمله فرمت I اضافه کنیم، باید آن را به عنوان یک فیلد مجزا در معادله‌ی طول دستور لحاظ کنیم.

معادله جدید جمع بیت‌ها به این شکل خواهد بود:

$$CC(2) + OPCODE(4) + Rd(3) + Rs1(3) + S(1) + imm(new) = 16$$

با جایگذاری مقادیر در معادله: $2 + 4 + 3 + 3 + 1 + \text{imm}(\text{new}) = 16$:

$$\rightarrow 13 + \text{imm}(\text{new}) = 16.$$

بنابراین، در این حالت نیز عرض جدید فیلد مقدار فوری برابر با ۳ بیت خواهد بود.

* آیا این مبادله (بیت S برای همه در برابر عرض کمتر مقدار فوری) ارزش دارد؟
خیر، از نظر معماری کامپیوتر ارزشش را ندارد.

چون دستوراتی که از فرمت I استفاده می کنند (مانند بارگذاری/ذخیره در حافظه LW/SW یا عملیات‌هایی مثل ADDI)، وابستگی شدیدی به گستره‌ی مقادیر فیلد فوری (آفست) دارند. کاهش عرض این فیلد از ۴ بیت به ۳ بیت، دامنه اعداد علامت‌دار قابل نمایش را به شدت محدود می کند (از $[-8, 7]$ به $[-4, 3]$). این محدودیت باعث می شود دسترسی به خانه‌های مختلف حافظه یا ساختن اعداد، نیازمند دستورات کمکی بسیار بیشتری باشد که کارایی برنامه را کاهش می دهد.

سوال پنجم:

پاسخ الف)

ADD R3, R1, R0

SUB R0, R1, R2

ADD.LT R3, R2, R0

اول فرض می کنیم R1 عدد بزرگتر هست. پس مقدارش رو با Register صفر جمع می کنیم و خروجی رو می ریزیم تو R3 خط دوم کار همون دستور مقایسه رو انجام می ده. R2 رو از R1 کم می کنیم و بیت مربوط به آپدیت فلوگ رو روشن

می‌ذاریم. جواب این تفریق تو R0 ریخته می‌شه تا دیتای بقیه Register ها خراب نشه و فقط فلگ‌ها بتونن آپدیت بشن. تو خط سوم پردازنده چک می‌کنه ببینه اگه شرط LT برقرار بود یعنی R1 از R2 کوچکتر بود میاد مقدار R2 رو می‌ریزه داخل R3 اگه این شرط برقرار نباشه این خط شبیه یه دستور NOP عمل می‌کنه و همون مقدار اولیه R1 داخل R3 باقی می‌مونه. تو پردازنده‌هایی مثل MIPS ما نمی‌تونیم دستورهای محاسباتی و عادی رو به صورت شرطی اجرا کنیم. برای همین وقتی می‌خوایم یه کاری مثل دور زدن یک انتساب رو انجام بدیم باید روی بیاریم به دستورهای Jump یعنی روند کار اینطوری می‌شه که اول مقایسه رو انجام می‌دیم بعد یک Jump شرطی می‌نویسیم تا اگه شرط ماکزیمم بودن برقرار نبود پردازنده بپره و خطوط بعدی رو اجرا نکنه. این طراحی هم باعث می‌شه تعداد خطوط کد بالا بره و هم به خاطر پرش کردن مسیر اجرای برنامه عوض بشه. وقتی پرش اتفاق میفته Pipeline پردازنده خالی می‌شه و پردازنده باید برای پر کردن دوباره اون کلی کلاک سایکل رو هدر بده.

پاسخ ب)

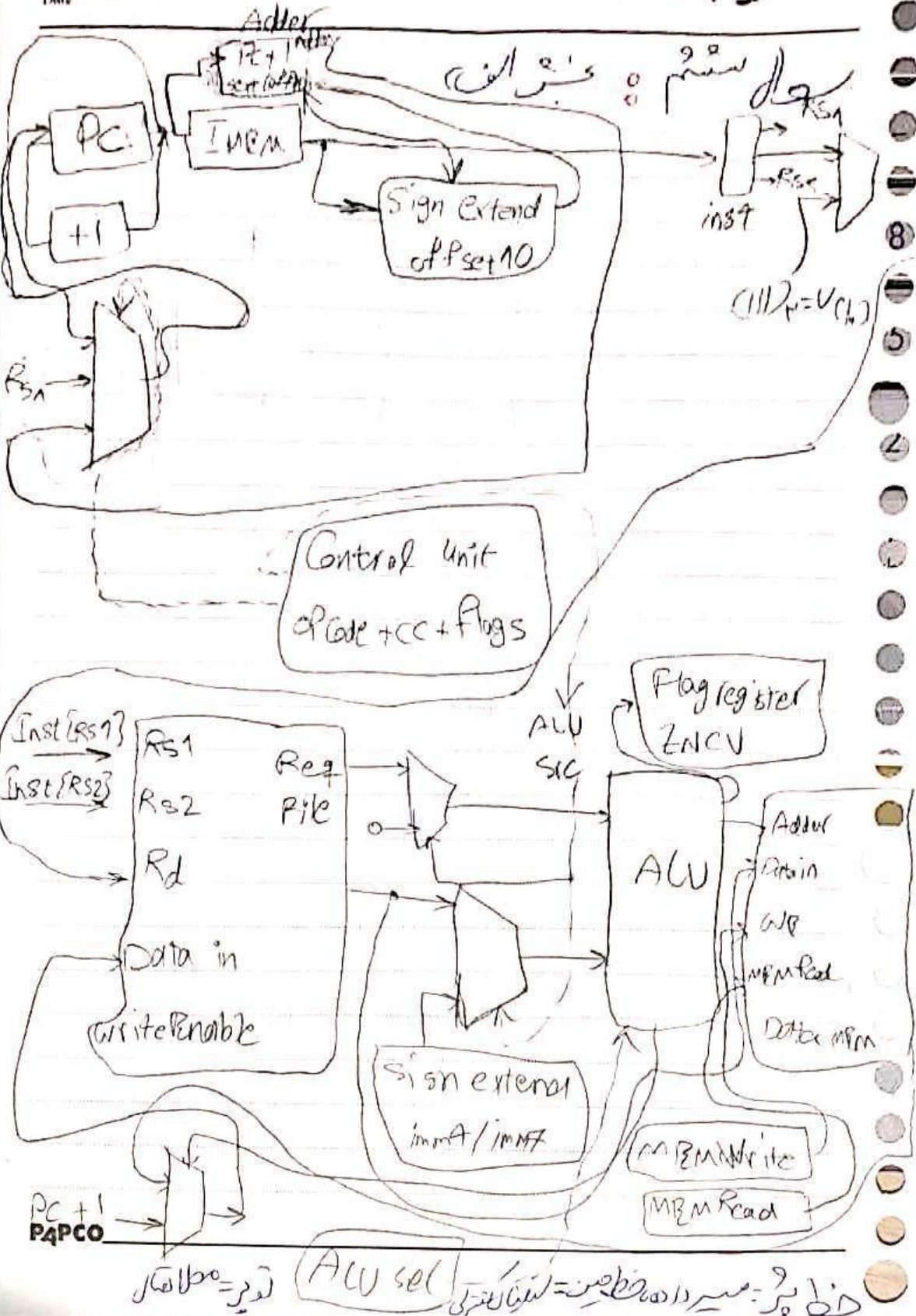
از نظر سخت افزاری اختصاص دادن دو بیت ثابت برای CC توی همه دستورها باعث میشه بیت های کمتری برای فیلدهای مهم دیگه مثل Opcode یا مقدار Immediate بمونه. توی یه پردازنده 16 بیتی مثل این، گرفتن دو بیت از تک تک دستورها به شدت جا رو برای بقیه فیلدها تنگ میکنه و نمیشه مقادیر ثابت بزرگتری رو آدرس دهی کرد. از نظر زمان اجرا و برنامه نویسی هم اگه یه بلاک کد شرطی طولانی داشته باشیم که شرطش برقرار نباشه، پردازنده مجبوره همه اون دستورها رو دونه دونه Fetch کنه و تازه بعد از چک کردن شرط اونا رو به NOP تبدیل کنه. این یعنی هدر دادن کلی کلاک برای خوندن دستوراتی که قرار نیست اجرا بشن، در حالی که توی

معماری Branch فقط با یه دستور پرش میشد از روی کل اون بلاک پرید و زمان اجرای برنامه رو خیلی کم کرد.

سوال ششم:

بخش الف)

"آرکس بزرگي ۱۳۰۴۰۱۳۰۴" "فایده سافت، کامپیوتر دکتوری"



بخش ب)

Subject: _____						
Date: _____						
	Reg Write	Mem Read	Mem Write	ALU-Set	WBSZ	ASZ
JAL	1	0	0	0	10	0
MOVI	1	0	0	0	00	0
ADD	1	0	0	001	00	0
ADDI	1	0	0	001	00	1

	Regbase	PCbase	Don't Care
JAL	01	01	
MOVI	00	00	
ADD	00	00	
ADDI	00	00	

سوال هفتم:

بخش الف)

1. ترجمه شبه‌دستور POP R4: با توجه به مثال ارائه‌شده در صورت سؤال برای دستور PUSH R5، در این معماری ابتدا داده در آدرسی که R6 (اشاره‌گر پشته) نشان می‌دهد ذخیره می‌شود و سپس اشاره‌گر پشته یک واحد کاهش می‌یابد. از آنجا که پشته به سمت آدرس‌های پایین رشد می‌کند، پس از عملیات پوش، R6 همواره به اولین خانه خالی پشته اشاره دارد. بنابراین، برای عملیات POP (خواندن و برداشتن از پشته)، باید این روند برعکس اجرا شود؛ یعنی ابتدا اشاره‌گر پشته یک واحد افزایش یابد تا به آخرین داده معتبر اشاره کند، و سپس آن داده از حافظه خوانده شده و در ثبات مقصد (R4) قرار گیرد.

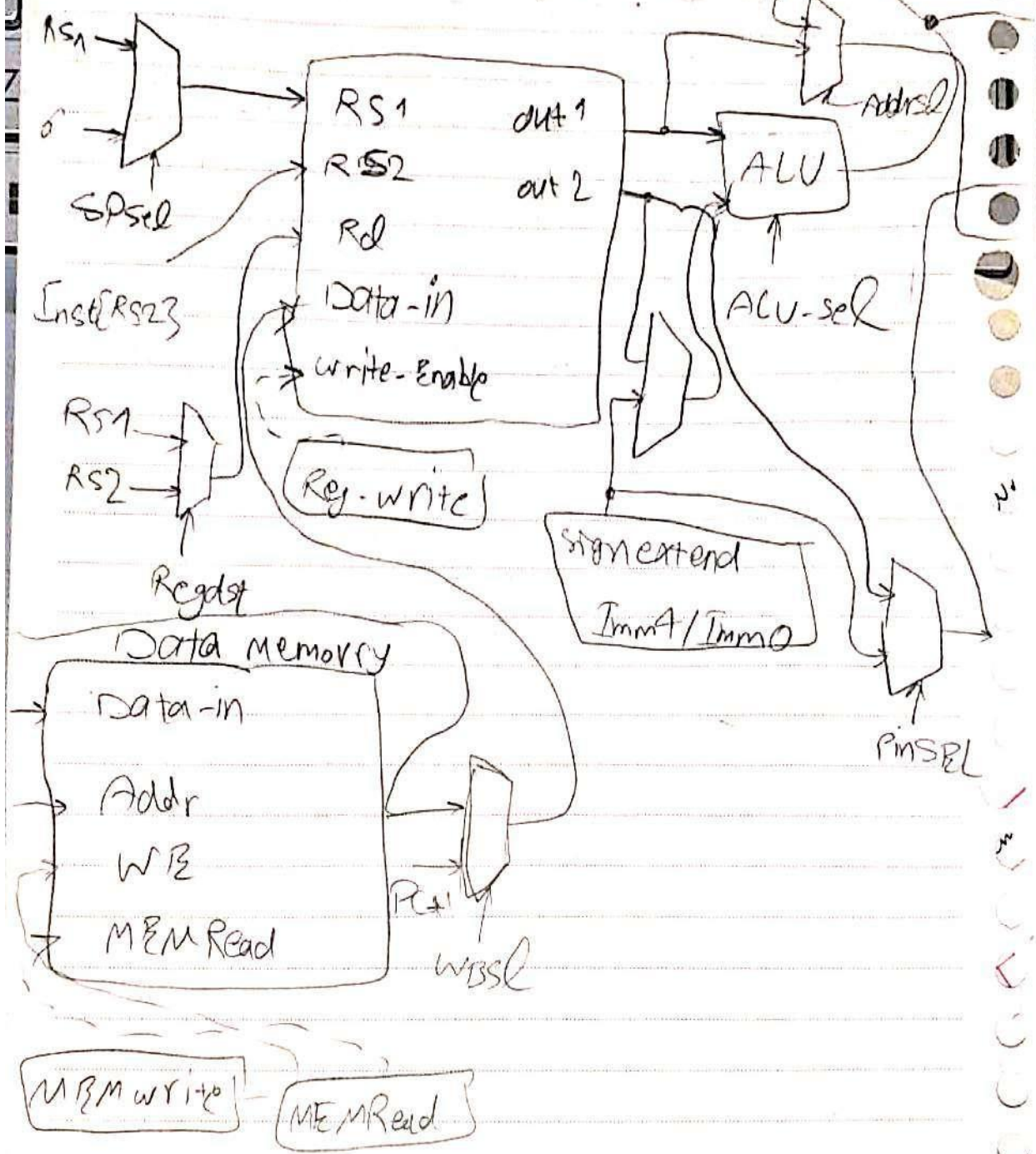
2. ترجمه شبه‌دستور PUSHI 40: در معماری PARS16، دستور ذخیره در حافظه (SW) تنها می‌تواند محتوای یک ثبات را در حافظه بنویسد و امکان نوشتن مستقیم مقادیر فوری در حافظه وجود ندارد. از این رو، برای قرار دادن عدد ۴۰ در پشته، ابتدا باید این عدد با استفاده از دستور MOVI (که مقادیر فوری تا ۷ بیت را پشتیبانی می‌کند و عدد ۴۰ در آن جا می‌شود) درون یک ثبات موقت بارگذاری شود. سپس عملیات PUSH دقیقاً مشابه الگو، روی آن ثبات موقت انجام می‌گیرد. (با فرض اولیه حل کردم)

بخش ب)

Subject: فایده استفاده از
 Date: دکتر باقری

"آرسن پزیری ۱۴۰۲/۱۰/۲۰"

سوال صحیح : ۰ : غلط : ۰



Subject: _____ Date: _____					
	Regwrite	MEMRead	MEMwrite	ALU sel	WBR
PUSH	1	0	1	00100	00100
PUSHI	1	0	1	00101	00101
	SPsel	Regdst	ALU sel	PinSEL	Addr sel
PUSH	1	2	2	0	1
PUSHI	1	2	2	1	1

بخش د) چون برای اجرای POP باید تو یک Clock Cycle دو تا رجیستر رو همزمان آپدیت کنیم یعنی یکی رجیستر مقصد که دیتای حافظه رو میگیره و یکی هم رجیستر R6 که باید مقدارش تغییر کنه به مشکل میخوریم Register File . مدار فعلی فقط یه دونه Write Port داره پس باید طراحی رو عوض کنیم و یه Write Port دوم با آدرس و دیتای مستقل بهش اضافه کنیم تا بتونه تو همون یک کلاک روی دو تا رجیستر مختلف بنویسه.